



30.1. Python: NumPy

"Numerical Computations with NumPy"

Previous Section:

[!\[\]\(c3d993ca47bfe2a953c700506ce31fa0_img.jpg\) 29. Python: Image Processing](#)

Contents:

[0. Introduce NumPy](#)

[0.1. What Can NumPy Be Used For?](#)

[0.2. Importing NumPy](#)

[0.3. Why NumPy Matters](#)

[1. Common Operations on NumPy](#)

[1.1. Creating Arrays and Matrices](#)

[1.2. Reshaping Arrays](#)

[1.3. Flattening Matrices](#)

[1.4. Random Arrays](#)

[1.5. Arrays Filled with Zeros or Ones](#)

[1.6. Arrays with Ranges](#)

[1.7. Indexing and Slicing](#)

[1.8. Common Types of Matrices](#)

[1.9. Common Operations on Matrices](#)

[1.10. Mathematical Operations](#)

[1.11. Logarithms](#)

[1.12. Linear Algebra](#)

[Summary](#)

References

When people first learn Python, they often focus on things they can immediately see—printing text, building applications, creating websites, or automating tasks. But behind many of the world's most important technologies lies something far more fundamental: numerical computation.

From scientific simulations and financial forecasting to artificial intelligence and machine learning, modern computing relies heavily on mathematics. And Python happens to be one of the most powerful languages for this purpose.

Fortunately, Python does not expect us to build every mathematical tool from scratch. Instead, it provides several specialized libraries designed specifically for numerical and symbolic computation. The three most famous are:

- **NumPy** – Fast numerical computing and array operations
- **SciPy** – Scientific computing and advanced mathematical algorithms
- **SymPy** – Symbolic mathematics and algebraic manipulation

Together, these libraries transform Python from a general-purpose programming language into a powerful mathematical and scientific toolkit.

In this chapter, we begin with the foundation of numerical computing in Python: **NumPy**.

0. Introduce NumPy

If Python had a language specifically designed for mathematics, that language would probably be NumPy.

NumPy (short for **Numerical Python**) is one of the most important libraries in the entire Python ecosystem. It provides efficient tools for working with numbers, arrays, matrices, vectors, and mathematical operations.

At first glance, NumPy may seem like a replacement for Python lists. But underneath, it is much more than that.

A NumPy array is designed specifically for numerical computation. It stores data more efficiently, performs calculations significantly faster, and provides built-in mathematical operations that would otherwise require many lines of code.

Because of this, NumPy has become the foundation of countless other libraries, including: Pandas; SciPy; Scikit-Learn; TensorFlow; PyTorch; OpenCV

In many ways, learning NumPy is the first step toward data science, machine learning, scientific computing, and artificial intelligence.

0.1. What Can NumPy Be Used For?

NumPy excels in situations involving large amounts of numerical data, including:

- Working with vectors and matrices
- Processing images and pixel data
- Handling time-series information
- Performing statistical calculations
- Solving linear algebra problems
- Running numerical simulations
- Supporting machine learning algorithms

Rather than operating on one number at a time, NumPy is designed to perform calculations across entire collections of data simultaneously, making it both elegant and efficient.

0.2. Importing NumPy

Before using NumPy, we need to import it:

```
import numpy as np
```

You might wonder why we use `np` instead of the full name `numpy`. The answer is simple: convention.

Technically, any alias would work:

```
import numpy as mymath
```

or:

```
import numpy as numbers
```

However, almost every Python programmer uses `np`. Following this convention makes code easier to read, easier to share, and easier to understand when working with tutorials, documentation, or other developers. While Python allows flexibility, consistency often prevents confusion. Think of `np` as the universal shorthand for NumPy.

0.3. Why NumPy Matters

The real strength of NumPy is not just that it stores numbers—it allows us to think mathematically. Instead of writing loops to process every value manually, we can perform operations on entire arrays with a single statement.

This ability to work with large collections of data efficiently is one of the reasons Python became a dominant language in science, engineering, data analytics, and machine learning.

1. Common Operations on NumPy

NumPy provides a surprisingly rich collection of operations that make numerical computation both powerful and enjoyable.

1.1. Creating Arrays and Matrices

Before we perform calculations, we first need data structures capable of storing numerical information efficiently. In NumPy, the most important structure is the **array**.

Think of an array as a mathematical container. It can store numbers in one dimension (similar to a list), two dimensions (a matrix), or even higher dimensions for complex scientific applications.

- **Creating Arrays:** The most common way to create an array is using

```
np.array()
```

```
import numpy as np

arr = np.array([10, 20, 30, 40, 50])
print(arr)
```

[10 20 30 40 50]

- Once created, we can inspect its properties:

```
print(len(arr))      # Number of elements
print(arr.size)      # Total number of elements
print(arr.shape)     # Shape of array
```

5

5

(5,)

The shape `(5,)` indicates a one-dimensional array containing five values.

- Creating Matrices

A matrix can be created by nesting arrays.

```
matrix = np.array([[1, 2, 3], [4, 5, 6]])
print(matrix)
```

[[1 2 3]

[4 5 6]]

Its shape becomes:

```
print(matrix.shape)
```

(2, 3)

Meaning: 2 rows; 3 columns



A Note About `np.matrix()`

NumPy also provides:

```
m = np.matrix("1 2 3; 4 5 6")
```

While it works, I generally do **not recommend** using `np.matrix()`. The semicolon syntax becomes difficult to read when matrices become large.

```
m = np.matrix("1 2 3 4 5;"  
              "6 7 8 9 10;"  
              "11 12 13 14 15")
```

Using nested arrays is usually much cleaner and more consistent with modern NumPy code.

1.2. Reshaping Arrays

Sometimes data is initially stored as an array but needs to be organized as a matrix. The `reshape()` method changes the dimensions without changing the data.

```
arr = np.array([1,2,3,4,5,6])  
  
matrix = arr.reshape(2,3)  
print(matrix)
```

```
[[1 2 3]  
 [4 5 6]]
```

1.3. Flattening Matrices

The reverse operation is `flatten()`.

```
flat = matrix.flatten()
print(flat)
```

```
[1 2 3 4 5 6]
```

This converts a matrix back into a one-dimensional array.

1.4. Random Arrays

Random values are extremely useful for simulations, testing, and Machine Learning.

- Generate random decimal values:

```
np.random.rand(3)
```

Possible output: [0.24 0.81 0.56]

- Generate random integers:

```
np.random.randint(1, 10, size=5)
```

```
[3 8 1 6 5]
```

1.5. Arrays Filled with Zeros or Ones

These are commonly used as placeholders and initialization values.

```
np.zeros(5)
```

```
[0. 0. 0. 0. 0.]
```

```
np.ones((2,3))
```

```
[[1. 1. 1.]  
 [1. 1. 1.]]
```

1.6. Arrays with Ranges

Creating sequences manually can be tedious.

- `arange()` works similarly to Python's `range()`:

```
np.arange(0, 10, 2)
```

```
[0 2 4 6 8]
```

- `linspace()` generates evenly spaced values.

```
np.linspace(0, 1, 5)
```

```
[0. 0.25 0.50 0.75 1.00]
```

This is particularly useful for plotting graphs and scientific calculations.

1.7. Indexing and Slicing

Once we have an array, we need a way to access its elements. For one-dimensional arrays, NumPy uses the familiar slicing syntax (***remember that the starting index is 0***):

`array[start:stop:step]`

Example:


```
arr = np.array([10, 20, 30, 40, 50])
```

```
print(arr[1])  
print(arr[1:4])  
print(arr[2:])  
print(arr[:3])  
print(arr[:,2])
```

20

[20 30 40]

[30 40 50]

[10 20 30]

[10 30 50]

For matrices, we specify:

`matrix[row, column]`

Example:

```
matrix = np.array([[1,2,3],  
                   [4,5,6],  
                   [7,8,9]])
```

```
# Row index 0
```

```
print(matrix[0])
```

```
# Element at row index 1, column index 2
```

```
print(matrix[1, 2])
```

```
# Entire rows
```

```
print(matrix[1, :])
```

```
# Entire columns
```

```
print(matrix[:, 1])
```

[1 2 3]

6

[4 5 6]

[2 5 8]

1.8. Common Types of Matrices

Linear Algebra introduces many special matrix structures. NumPy provides convenient functions for most of them.

- **Square Matrix:** A matrix with equal numbers of rows and columns.

```
square = np.array([[1,2], [3,4]])  
print(square)
```

```
[[1 2]  
 [3 4]]
```

- **Diagonal Matrix:** All non-diagonal elements are zero.

```
diag = np.diag([1,2,3])  
print(diag)
```

```
[[1 0 0]  
 [0 2 0]  
 [0 0 3]]
```

- **Identity Matrix:** A special diagonal matrix where all diagonal entries are 1.

```
identity = np.identity(3)  
print(identity)
```

```
[[1. 0. 0.]  
 [0. 1. 0.]  
 [0. 0. 1.]]
```

- **Triangular Matrix:** This is a square matrix where either the upper half or lower half is only zeros

```
A = np.array([[1,2,3],
              [4,5,6],
              [7,8,9]])
```

```
print(np.triu(A))
```

```
[[1 2 3]
 [0 5 6]
 [0 0 9]]
```

```
print(np.tril(A))
```

```
[[1 0 0]
 [4 5 0]
 [7 8 9]]
```

- **Transpose Matrix:** Rows become columns and columns become rows.

```
A = np.array([[1,2,3],
              [4,5,6]])
```

```
print(np.transpose(A))
```

```
[[1 4]
 [2 5]
 [3 6]]
```

1.9. Common Operations on Matrices

Most mathematical operators work directly on NumPy arrays.

```
A = np.array([1,2,3])
B = np.array([4,5,6])
```

- Addition

```
print(A + B)
print(np.add(A, B))
```

```
[5 7 9]
[5 7 9]
```

- Subtraction

```
print(A - B)
print(np.subtract(A, B))
```

```
[-3 -3 -3]
[-3 -3 -3]
```

- Division

```
print(A / B)
print(np.divide(A, B))
```

```
[0.25 0.4 0.5]
[0.25 0.4 0.5]
```

- Element-wise Multiplication: Each element multiplies its corresponding element.

```
print(A * B)
print(np.multiply(A, B))
```

```
[4 10 18]
[4 10 18]
```

- Scalar Multiplication

```
print(A * 10)
print(np.multiply(A, 10))
```

```
[10 20 30]
```

```
[10 20 30]
```

- Dot Product: The dot product is one of the most important operations in mathematics, physics, and Machine Learning.

```
print(np.dot(A, B))
```

```
32
```

Explanation: $(1 \times 4) + (2 \times 5) + (3 \times 6) = 32$

- Outer Product: Instead of producing a single value, every element is multiplied with every other element.

```
print(np.outer(A, B))
```

```
[[ 4  5  6]
```

```
 [ 8 10 12]
```

```
 [12 15 18]]
```

1.10. Mathematical Operations

NumPy also includes many mathematical functions.

- Constants

```
print(np.pi)
print(np.e)
```

```
3.141592653589793
2.718281828459045
```

- Sum, Mean and Median

```
data = np.array([10, 20, 30, 40, 50])

print(np.sum(data))
print(np.mean(data))
print(np.median(data))
```

```
150
30.0
30.0
```

- Minimum and Maximum

```
print(np.min(data))
print(np.max(data))
```

```
10
50
```

- Variance and Standard Deviation: These are fundamental concepts in Statistics and Data Science.

```
print(np.var(data))
print(np.std(data))
```

```
200.0
14.142135623730951
```

- Square Root and Absolute Value

```
print(np.sqrt(16))  
print(np.abs(-25))
```

```
4.0  
25
```

- Power and Exponential

```
print(np.power(2, 5))  
print(np.exp(1))
```

```
32  
2.718281828459045
```

1.11. Logarithms

```
print(np.log(10))  
print(np.log2(8))  
print(np.log10(100))
```

```
2.302585092994046  
3.0  
2.0
```

1.12. Linear Algebra

One of NumPy's greatest strengths is its Linear Algebra module (`np.linalg`). Many Machine Learning algorithms are essentially sophisticated Linear Algebra systems.

- Determinant: A determinant describes certain properties of a square matrix.

```
A = np.array([[1, 2, 3],
               [3, 4, 3],
               [6, 6, 7]])
```

```
print(np.linalg.det(A))
```

-14.000000000000004

- Inverse Matrix: The inverse matrix is analogous to division for matrices.

```
A_inv = np.linalg.inv(A)
print(A_inv)
```

```
[[-0.71428571 -0.28571429  0.42857143]
 [ 0.21428571  0.78571429 -0.42857143]
 [ 0.42857143 -0.42857143  0.14285714]]
```

- Solving Systems of Linear Equations - Suppose:

$$\begin{cases} x + y = 5 \\ 2x + y = 8 \end{cases}$$

We can solve it directly:

```
A = np.array([[1, 1], [2, 1]])
B = np.array([5, 8])

solution = np.linalg.solve(A, B)
print(solution)
```

[3. 2.] → x = 3; y = 2

- Eigenvalues and Eigenvectors: One of the most important concepts in Machine Learning, Computer Vision, and Data Science.

- Eigenvalues describe how much a transformation stretches space, while eigenvectors describe the directions that remain unchanged by that transformation.
- And surprisingly, techniques such as **PCA (Principal Component Analysis)**, Google's original **PageRank**, image compression, recommendation systems, and many Machine Learning algorithms are built upon these ideas.

```
A = np.array([[4, 2, 3],
              [1, 3, 3],
              [2, 6, 1]])

eigenvalues, eigenvectors = np.linalg.eig(A)

print("Eigenvalues:\n", eigenvalues)
print("Eigenvectors:\n", eigenvectors)
```

Eigenvalues:

```
[ 8.10230638  2.43353975 -2.53584613]
```

Eigenvectors:

```
[[-0.65858851 -0.90421728 -0.26885122]
```

```
[-0.47313086  0.35729958 -0.42092301]
```

```
[-0.5851567  0.23394042  0.86633875]]
```

Summary

NumPy is the foundation of scientific computing in Python. It specializes in:

- Arrays and matrices
- Linear algebra
- Fast numerical computations
- Statistical calculations
- Mathematical operations

Think of NumPy as the library that efficiently stores and manipulates numerical data.

References

<https://numpy.org/doc/stable/>

<https://www.geeksforgeeks.org/numpy/python-numpy/>

The next section covers the SciPy library:

[`</>` 30.2. Python: SciPy](#)